

Security Audit Report

ExamForge AI | Generated: 2026-08-02

Executive Summary

The security audit identified 12 findings across 12 categories, including 2 Critical, 4 High, 4 Medium, and 2 Low severity issues. The most critical findings are: no Content Security Policy configured (leaving the application vulnerable to XSS, clickjacking, and data exfiltration), and server actions lacking authorization checks (allowing any authenticated user to modify any notification, school, or profile). The Supabase Auth implementation is generally sound, using `getUser()` correctly in middleware and the SSR client pattern properly.

#	Finding	Severity	Category
1	No Content Security Policy configured	Critical	CSP
2	Server actions lack authorization checks	Critical	Auth
3	Auth callback open redirect via next parameter	High	Redirect
4	Password change does not verify current password	High	Auth
5	No rate limiting on Next.js API routes	High	Rate Limit
6	API routes excluded from middleware auth guard	High	Middleware
7	No security headers on Next.js API responses	Medium	Headers
8	<code>getSession()</code> used after <code>getUser()</code> in API routes	Medium	JWT
9	No CSRF protection on billing webhook proxy	Medium	CSRF
10	<code>updateSchoolAction</code> lacks auth, validation, role check	Medium	Server Actions
11	<code>.env</code> contains anon key (should be <code>.env.local</code>)	Low	Config
12	Environment variables properly namespaced	Pass	Config

Critical Findings

F-8: No Content Security Policy Configured

No Content Security Policy is configured anywhere in the Next.js application. This leaves the application vulnerable to inline script injection, unauthorized resource loading, clickjacking via `iframe` embedding, and data exfiltration via injected scripts. A CSP header should be added in `next.config.ts` or via middleware, specifying allowed sources for scripts, styles, images, fonts, and connections (including the Supabase domain and WebSocket endpoint).

Recommended Fix: Add CSP headers in `middleware.ts` or `next.config.ts`:

```
default-src 'self'; script-src 'self' 'unsafe-eval' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' blob: data: https://pzfnptrrnxkgodclyhft.supabase.co; font-src 'self' data;; connect-src 'self' https://pzfnptrrnxkgodclyhft.supabase.co wss://pzfnptrrnxkgodclyhft.supabase.co; frame-ancestors 'none'; base-uri 'self'; form-action 'self';
```

F-2: Server Actions Lack Authorization Checks

Multiple server actions perform data mutations without verifying the user is authorized: `markNotificationReadAction` allows any user to mark any notification as read;

deleteNotificationAction allows any user to delete any notification; updateSchoolAction allows any user to update any school; deactivateSchoolAction allows any user to deactivate any school; updateUserProfile allows any user to update any profile. These actions should verify the user identity and role before performing mutations. RLS policies on the Supabase side should also enforce these constraints, but the application should not rely solely on RLS.

High Severity Findings

F-5: Auth Callback Open Redirect

The auth callback route reads the "next" query parameter and uses it directly in a redirect without validation. An attacker could craft a URL like `/api/auth/callback?code=xxx&next;=//evil.com` which could redirect to unexpected internal paths or bypass onboarding flows. The fix is to validate that the next parameter starts with `/` and does not start with `//`.

F-3: Password Change Without Current Password Verification

The settings page password change function checks that the current password field is not empty, but never actually verifies the current password against Supabase. The `supabase.auth.updateUser()` call will update the password regardless because the user already has an active session. The fix is to use `signInWithPassword()` before allowing the update.

F-9: No Rate Limiting on Next.js API Routes

The Next.js API routes that proxy to Edge Functions have no rate limiting. An attacker can brute-force AI endpoints, flood billing checkout/refund endpoints, spam the search endpoint, or brute-force login/signup. While Edge Functions have their own rate limiting, the Next.js proxy layer is unprotected. For production, use Redis-backed rate limiting (e.g., Upstash) or an in-memory rate limiter with configurable limits per endpoint.

F-4: API Routes Excluded from Middleware Auth Guard

The middleware explicitly skips auth checks for API routes. This means every API route must independently implement auth checks. While most routes do check auth, the billing webhook route does not verify the Flutterwave signature. The Next.js proxy should verify the webhook signature or add rate limiting before forwarding to the Edge Function.

Positive Findings

The codebase demonstrates generally sound security practices: `getUser()` is correctly used in middleware for JWT validation (not `getSession()`); the SSR client is properly configured with cookie handlers; the browser client uses a singleton pattern; signup forces role to "student" preventing role escalation; the service role key is never exposed to the client; and Edge Functions have comprehensive security headers including HSTS for production.

Priority Remediation Order

Priority	Finding	Action
P0	F-8: No CSP	Add Content Security Policy headers

Priority	Finding	Action
P0	F-2: Server action auth	Add authorization checks to all server actions
P1	F-5: Open redirect	Validate next parameter in auth callback
P1	F-3: Password verify	Verify current password before allowing change
P1	F-9: Rate limiting	Add rate limiting to Next.js API routes
P1	F-4: API auth guard	Verify webhook signatures or add rate limiting
P2	F-7: Security headers	Add X-Content-Type-Options, X-Frame-Options, etc.
P2	F-8: getSession pattern	Add session validity check after getUser()
P2	F-9: CSRF webhook	Add origin verification to webhook proxy
P3	F-10: .env.local	Move anon key from .env to .env.local